



PROYECTO DE PROGRAMACIÓN IMPERATIVAS – GUIA #07
Estación de Envíos y Paquetes 'Envía Ya'

Estudiantes:

ANDREA HERRERA PATIÑO – 202651161-2724
ANDERSON GEOVANNY LOPEZ RAMIREZ – 200860206-2724

Docente:

Ing. HUBER ANTONIO ARBOLEDA SOLARTE

750012C – FUNDAMENTOS DE PROGRAMACIÓN IMPERATIVA
UNIVERSIDAD DEL VALLE SECCIONAL BUGA
31 JULIO DEL 2026
GUADALAJARA DE BUGA

INFORME PROYECTO DE PROGRAMACIÓN IMPERATIVAS – GUIA #07

Estación de Envíos y Paquetes 'Envía Ya'

Resumen del Proyecto

El presente proyecto, consiste en el desarrollo de un software de escritorio para la agencia postal 'EnvíaYa', diseñado para registrar paquetes en la bahía de carga, evaluar su peso y detectar encomiendas prioritarias. El programa cuenta con una interfaz gráfica construida en Tkinter e implementa un motor de procesamiento de datos exclusivamente recursivo, cumpliendo con la restricción técnica de no utilizar ciclos for o while.

EXPLICACIÓN TÉCNICA

1. Función: `calcular_peso_total(lista_paquetes)`

Esta función recursiva tiene como objetivo calcular la suma de los kilogramos de todos los paquetes almacenados en el manifiesto.

- **Caso Base:** La recursión se detiene cuando la lista de paquetes está vacía (if `lista_paquetes == []:`). En este punto, no hay más paquetes que pesar, por lo que la función retorna 0.
- **Caso Recursivo:** Mientras la lista contenga elementos, la función extrae el peso del primer paquete de la lista (`lista_paquetes[0][1]`) y lo suma al resultado de llamarse a sí misma pasándole el resto de la lista (`calcular_peso_total(lista_paquetes[1:])`).

Diagrama de Flujo / Traza de Llamadas (Ejemplo con 2 paquetes: 10kg y 5kg):

1. `calcular_peso_total([[A, 10, ...], [B, 5, ...]])` -> Retorna `10 + calcular_peso_total([[B, 5, ...]])`.
2. `calcular_peso_total([[B, 5, ...]])` -> Retorna `5 + calcular_peso_total([])`.
3. `calcular_peso_total([])` -> Alcanza el caso base y retorna 0.
4. Desapilado: $5 + 0 = 5$ -> $10 + 5 = 15$. El peso total es 15 kg.

2. Función: `contar_paquetes_fragiles(lista_paquetes)`

Esta función recursiva retorna la cantidad de paquetes registrados que requieren manipulación frágil.

- **Caso Base:** Al igual que en la función anterior, si la lista de paquetes está vacía (if `lista_paquetes == []:`), la función retorna 0 indicando que no hay paquetes frágiles por contar.
- **Casos Recursivos:**
 - **Si es frágil:** Si el atributo de fragilidad del primer paquete es verdadero (if `lista_paquetes[0][3]:`), la función suma 1 y hace el llamado recursivo con el resto de la lista (`1 + contar_paquetes_fragiles(lista_paquetes[1:])`).

- **Si NO es frágil:** Si el paquete no es frágil, la función no suma nada y simplemente continúa la evaluación con el resto de la lista (`contar_paquetes_fragiles(lista_paquetes[1:])`).

Diagrama de Flujo / Traza de Llamadas (contar_paquetes_fragiles) (Ejemplo con P1 frágil y P2 normal):

1. `contar_paquetes_fragiles([P1(frágil), P2(normal)])` -> Es frágil, retorna `1 + contar_paquetes_fragiles([P2(normal)])`.
2. `contar_paquetes_fragiles([P2(normal)])` -> No es frágil, retorna `contar_paquetes_fragiles([])`.
3. `contar_paquetes_fragiles([])` -> Caso base, retorna 0.
4. Desapilado: `0 -> 1 + 0 = 1`. Total frágiles: 1.

3. Función: `obtener_paquetes_sobrepeso(lista_paquetes, peso_limite)`

Esta función recursiva genera una nueva lista con los códigos de los paquetes cuyo peso supera un valor límite indicado por el usuario.

- Caso Base: Si la lista se encuentra vacía (`if lista_paquetes == []:`), retorna una lista vacía `[]`, lo que sirve como base para construir la lista final.
- Casos Recursivos:
 - Si supera el peso límite: Si el peso del primer paquete evaluado es mayor al límite (`if lista_paquetes[0][1] > peso_limite:`), se toma el código del paquete (`lista_paquetes[0][0]`) y se concatena en forma de lista con el resultado del llamado recursivo del resto de los paquetes (`[lista_paquetes[0][0]] + obtener_paquetes_sobrepeso(lista_paquetes[1:], peso_limite)`).
 - Si NO supera el peso límite: Si el paquete está dentro del peso permitido, se ignora su código y se avanza al siguiente elemento realizando el llamado recursivo con el resto de la lista (`obtener_paquetes_sobrepeso(lista_paquetes[1:], peso_limite)`).

Diagrama de Flujo / Traza de Llamadas (Ejemplo límite 20kg: P1(25kg), P2(10kg)):

1. `obtener_paquetes_sobrepeso([P1(25kg), P2(10kg)], 20)` -> `25 > 20`, retorna `[Código_P1] + obtener_paquetes_sobrepeso([P2(10kg)], 20)`.
2. `obtener_paquetes_sobrepeso([P2(10kg)], 20)` -> `10 < 20`, retorna `obtener_paquetes_sobrepeso([], 20)`.
3. `obtener_paquetes_sobrepeso([], 20)` -> Caso base, retorna `[]`.
4. Desapilado: `[] -> [Código_P1] + [] = [Código_P1]`. Lista final generada.

Enlace Github: <https://github.com/xakir89/ProyectoFinal>